

WEEK 7

Sequential Data & Probabilistic Reasoning



From Statistical Foundations
to Modern Deep Learning

ANOVA

RNN

Transformers



Raw Waveform



Statistical Distribution

The Taxonomy of Time-Ordered Data

Slide 2 · Defining Sequence Data Types



Time-Series

Continuous numerical values at fixed intervals

Temperature sensors · Stock prices



Event Streams

Discrete, irregular occurrences

Log files · User clicks · Transactions



Symbolic Sequences

Discrete tokens where position determines meaning

DNA · Natural Language · Music notation



Spatiotemporal

Moving coordinates in space and time

GPS trails · Video frames · Sensor grids

Hidden Markov Models (HMM)

Slide 9 · Probabilistic Reasoning

The Concept

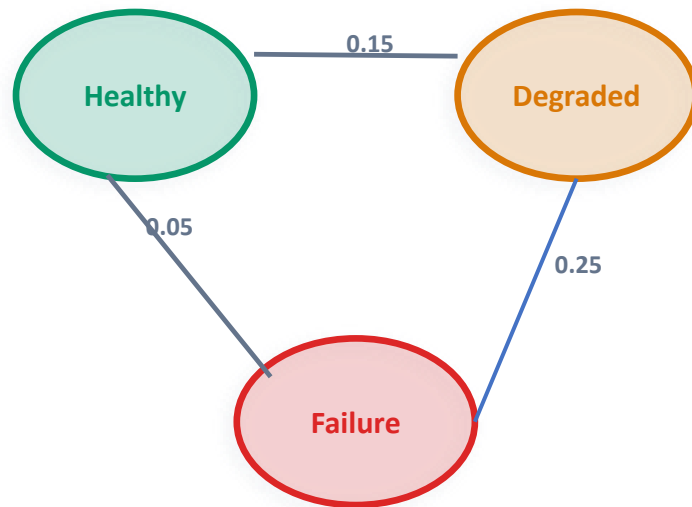
We observe Sensor data but want to know the Hidden State — system health, structural integrity, or failure mode.

Transition Probabilities

How likely is it to stay 'Healthy' vs. transitioning to 'Degraded' or 'Failure'?

Emission Probabilities

What does each hidden state 'look like' in the observable data?
What sensor signature indicates Failure?



Observations (Sensor data):

0.1

0.3

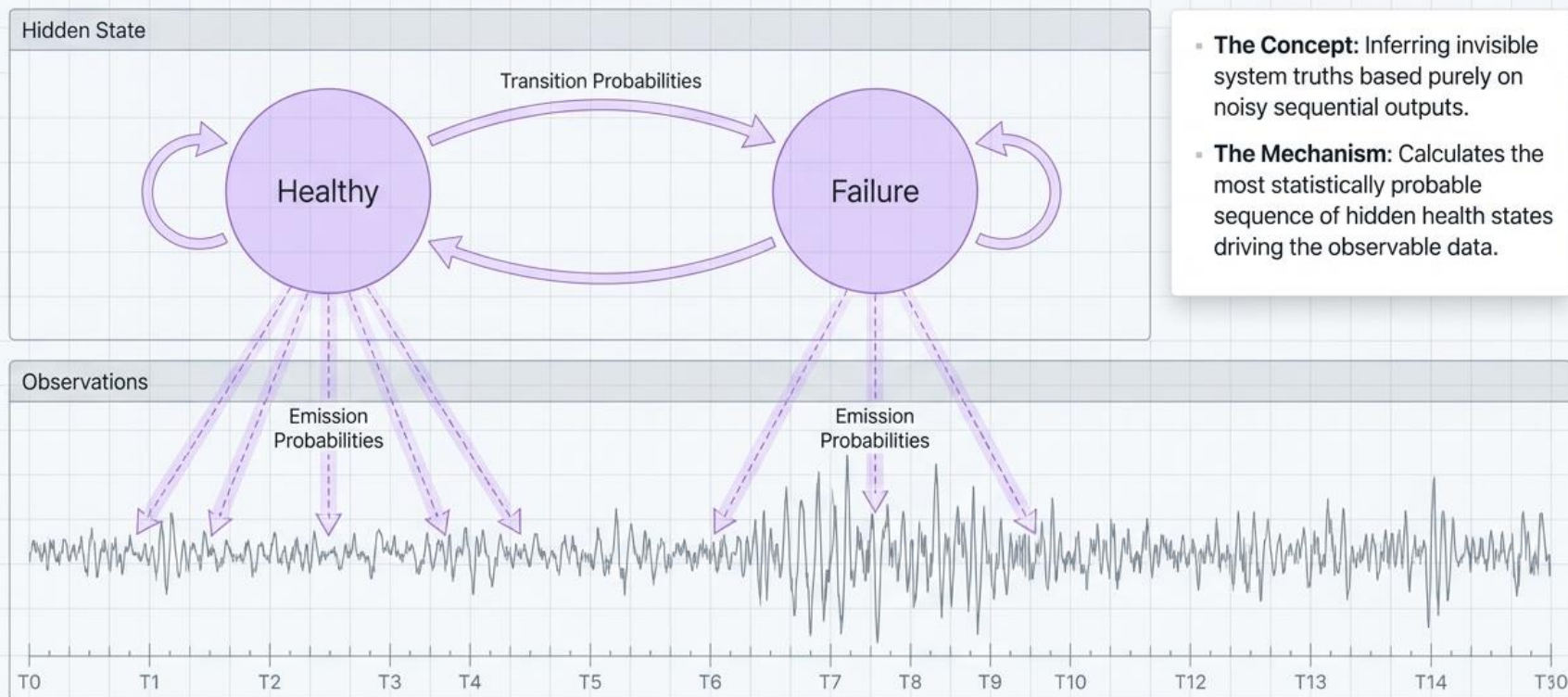
0.6

0.9

0.8

1.2

Probabilistic Reasoning: Hidden Markov Models (HMM)



Modeling Variance Across Time (ANOVA)

Slide 3 · Statistical Foundations

The Concept

Used when the same subject is measured multiple times. It asks: Is the mean difference across time points statistically significant?

Example

Testing user response latency across three consecutive system updates.

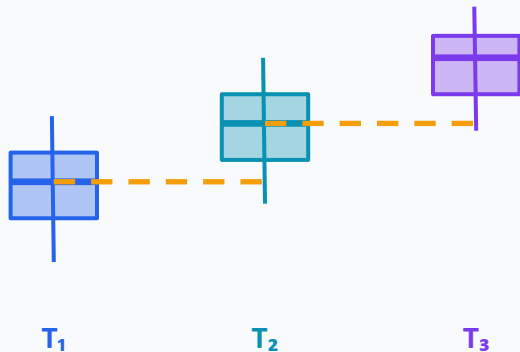
Variance Partition

Total Variance = Between-Subjects + Within-Subjects

F-Statistic Formula

$$F = MS_{\text{time}} / MS_{\text{error}}$$

Latency Across Updates



Processing the Chain: Recurrent Neural Networks

Slide 4 · Deep Learning I

Hidden State Equation

$$h_t = \sigma(W_h \cdot h_{(t-1)} + W_x \cdot x_t + b)$$

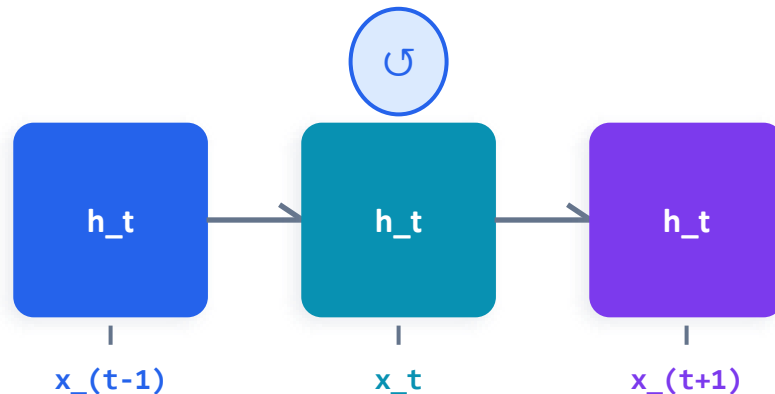
The Logic

Processes inputs one by one, passing a hidden state (memory) to the next step in the sequence.

The Problem: Vanishing Gradient

In long sequences, the memory of the first token disappears before reaching the end.

Unrolled RNN — Memory flows left to right



⚡ Solution: LSTM & Transformers

Long Short-Term Memory (LSTM) adds gates to control what is remembered or forgotten, addressing the vanishing gradient problem that plagues vanilla RNNs.

Parallel Context: The Transformer Architecture

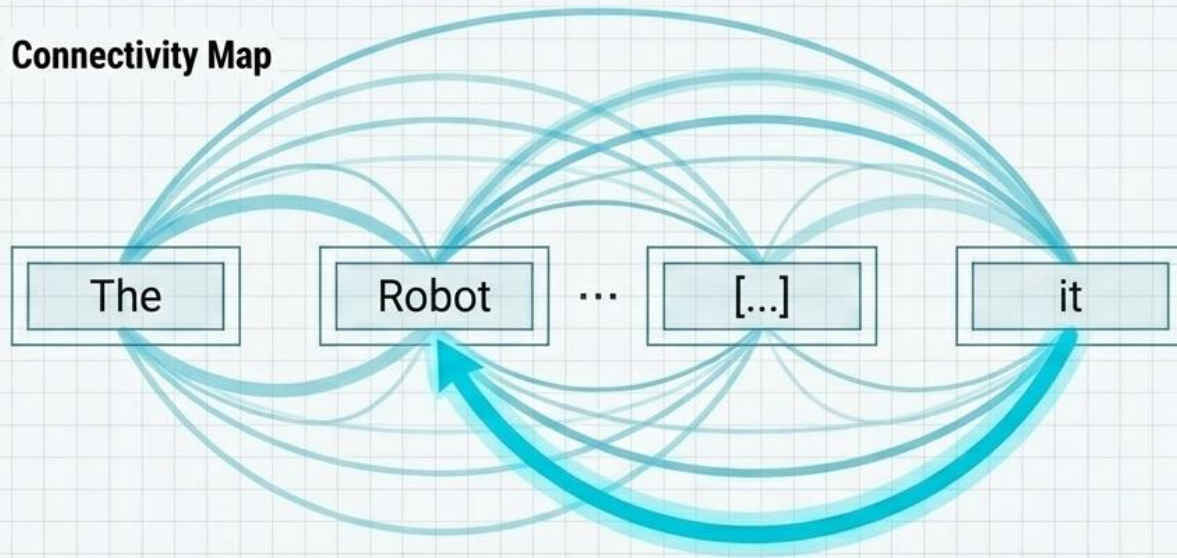
The Logic:

Eliminates recursion entirely. Analyzes the full sequence simultaneously.

Self-Attention: Every token mathematically 'attends' to every other token to establish relevance instantly.

Advantage: Highly parallelizable on GPUs; seamlessly handles long-range dependencies.

Connectivity Map



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q * K^T}{\sqrt{d_k}}\right) * V$$

Parallel Context: The Transformer & Self-Attention

Slide 5 · Deep Learning II

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

No Recursion

Looks at the entire sequence at once — no serial bottleneck.

Self-Attention

Every token attends to every other token to find relevance across the full context.

GPU Parallelism

Highly parallelizable on GPUs; handles long-range dependencies perfectly.

The

Robot

lost

it

s

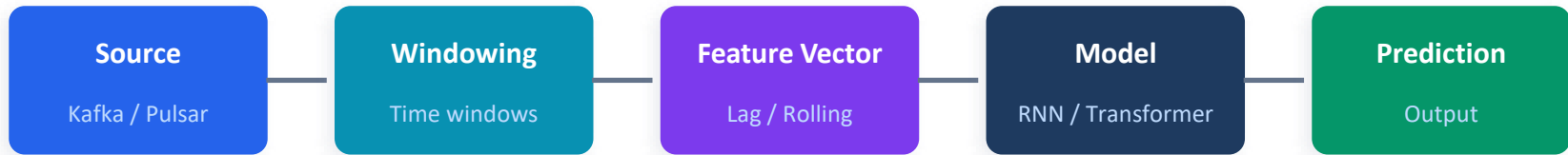
charge

← Self-Attention:
"it" refers to
"Robot"

Foundation of GPT, BERT, and all modern LLMs

The Sequential Pipeline Architecture

Slide 6 · AI Pipeline Design



Ingestion

High-velocity timestamp handling via Kafka or Apache Pulsar. Each event carries a precise timestamp for ordering.



Feature Engineering

Lag Features: $x_{(t-n)}$
Rolling Aggregates: Mean/Std of the last 10 minutes.
State Store: Redis, Flink State.



State Store

The system's memory lives in a persistent store (Redis, Flink State). Critical for maintaining context across inference calls.

Implementation: Sliding Windows in Python

Slide 7 · Practical Code

feature_engineering.py

```
import pandas as pd

# 1. Rolling Statistics (Statistical Feature)
df['temp_ma'] = df['temp'].rolling(window=5).mean()

# 2. Lagging (Sequential Feature for RNNs)
df['prev_val'] = df['temp'].shift(1)

# 3. Differencing (Stationarity for ANOVA)
df['delta'] = df['temp'].diff()
```

Rolling / Smoothing

Computes a moving average over a window of 5 samples — reduces noise.

Lagging / History

Creates a shifted version of the column — provides the model its prior value.

Differencing / Change

Computes the delta between consecutive rows — key for stationarity in ANOVA.

Reasoning Over Time: Temporal Uncertainty

Slide 8 · Probabilistic Foundations



Decay of Certainty

The further into the future we predict, the higher the variance. Prediction intervals widen exponentially.



Sensor Noise

Distinguishing between a real signal change and random jitter is a core challenge in real-time inference.

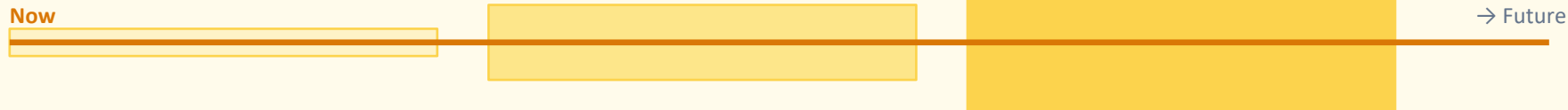


Missing Data

How does the system reason when a packet is dropped? Imputation strategies must be part of your pipeline.

Forecast Fan Chart — Uncertainty Grows Over Time

Now



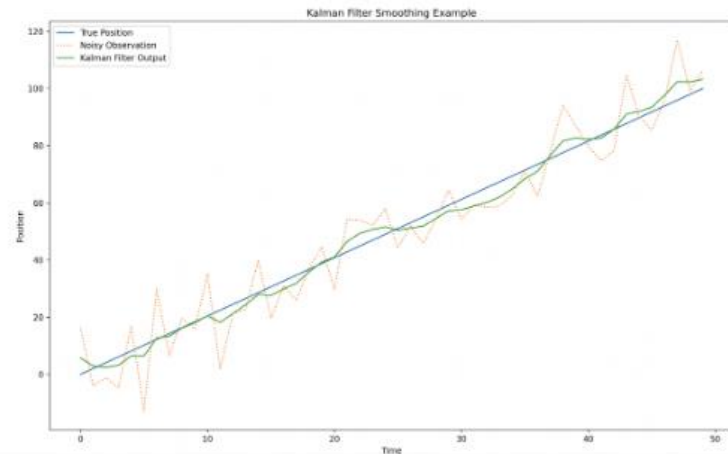
Smoothing: The Kalman Filter

Usage: GPS, AVs, Radar.

Predict: Where *should* it be based on physics?

Observe: Where *does* the sensor say it is?

Update: Compute a weighted average to find the most likely position.



State Estimation: The Kalman Filter

Slide 10 · Smoothing the Stream

1

Predict

Where should the system be? Use the model's prior estimate to project forward.

2

Observe

Where does the sensor say it is? Read the noisy measurement from the hardware.

3

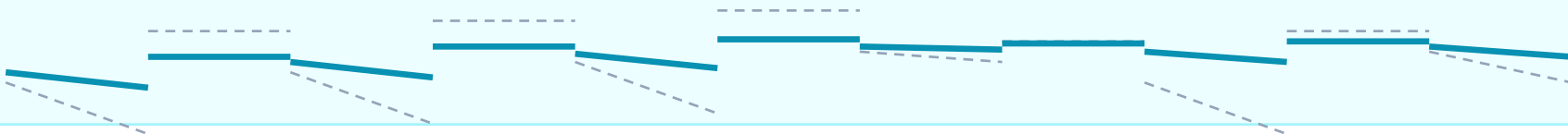
Update

Compute the Kalman Gain — the weighted average between prediction and measurement.

Applications: 🚗 Autonomous Vehicles • 📍 GPS Tracking • ✈️ Aerospace Navigation • 🤖 Robotics

Signal Visualization

— — Noisy Raw Signal — — Kalman Filtered Output



Lambda vs. Kappa Architectures

Slide 11 · System Design

λ Lambda Architecture

Batch Layer

High accuracy, processes all historical data. Run periodically (hours/days).

Speed Layer

Low latency, processes recent data only. Accepts some inaccuracy for real-time delivery.

Serving Layer

Merges batch views with real-time delta. Answers all incoming queries.

κ Kappa Architecture

Everything is a Stream

Both real-time and historical data flow through one unified stream pipeline.

Stream Replay

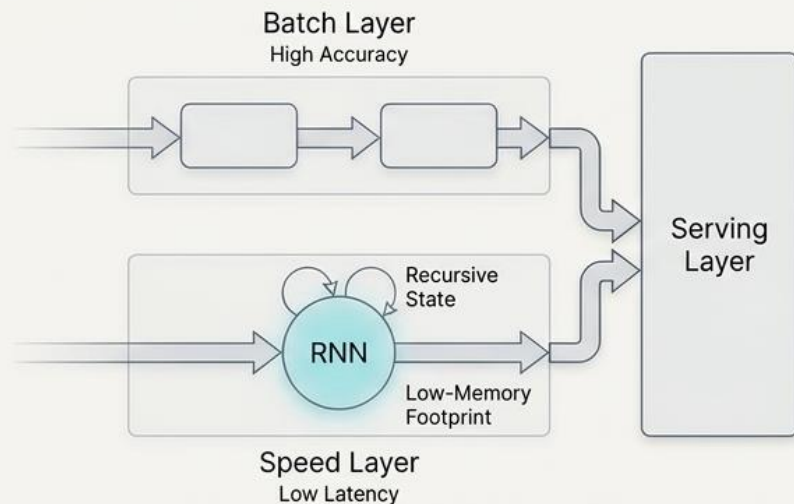
Re-run historical analysis by 'replaying' the event log — no separate batch system.

Simplicity

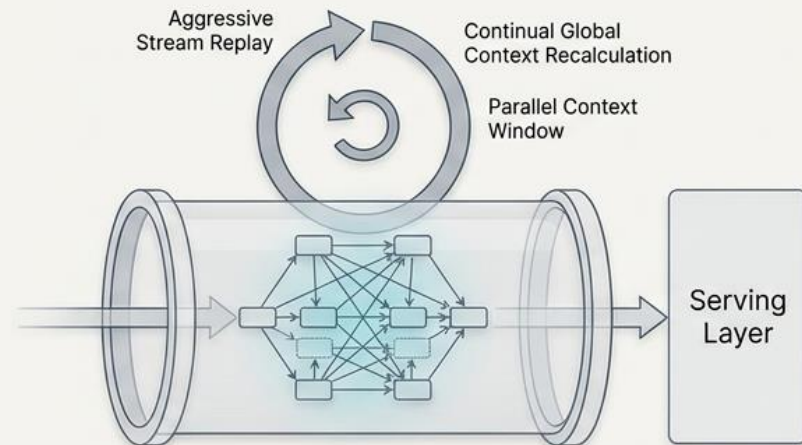
Fewer systems to maintain. Ideal for Transformer pipelines where context windows are replayed per prediction.

Architectural Synthesis: Lambda vs. Kappa Infrastructures

Lambda Architecture



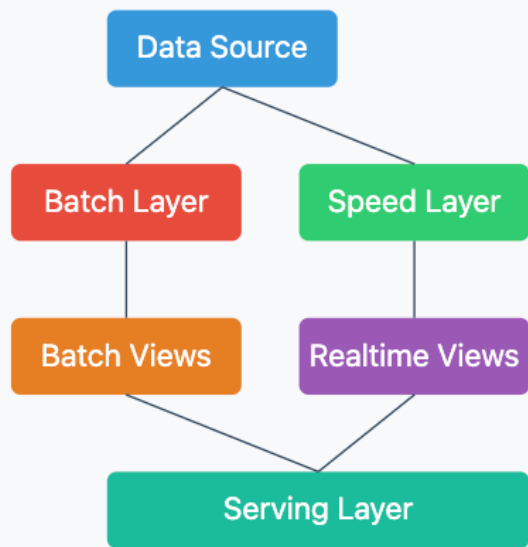
Kappa Architecture



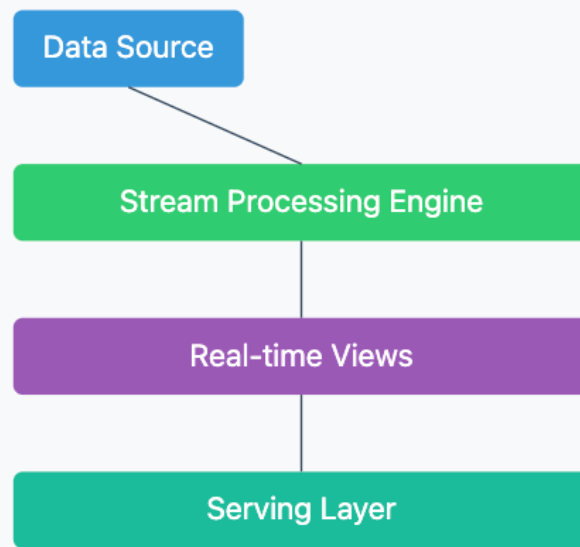
Algorithms dictate infrastructure. The recursive memory of an RNN integrates seamlessly into the stateful loops of a Lambda Speed Layer. A Transformer's demand for simultaneous global context forces a Kappa stream-replay architecture.

Lambda vs Kappa Architecture

Lambda Architecture



Kappa Architecture



Key Difference: Single vs Dual Processing Path

Week 7

Wrap-Up

Next: Speech Recognition
— The ultimate test of
sequential reasoning

Model Selection Guide

ANOVA

Statistics

Simple significance testing across time points or model versions

RNN / LSTM

Deep Learning

Low-latency, serial memory tasks — streaming inference, real-time sensors

Transformer

Deep Learning

High-accuracy, global context tasks — NLP, complex sequence classification



Project Milestone

Your project must now account for temporal drift and state management. Ensure your pipeline handles missing data, windowing, and uncertainty quantification.